

Group	Loadwatchers - 12	**Application**	LoadWatch
Members	Muaaz (Group Leader), Monwabisi, Jadey, Rahil		
Modules	Programming 3D (PROG7314) / Open-Source Coding Intermediate (OPSC7312)		
Date	24 August 2026	**Referencing**	1994 words excluding diagrams, headings and tables

1. Introduction

LoadWatch is a native Android application that answers one question for a South African household: *is my power on, when does it go off, and when does it return?* The user selects a suburb, the application retrieves that area's schedule from a REST API this group builds and hosts, and the home screen shows current supply state, the next event and a live countdown.

This report analyses three Android applications already serving that need: **ESP "EskomSePush"**, **Load Shedding Notifier** and **My Smart City**. Each is examined for purpose, strengths and weaknesses, probable technical implementation and interface architecture, using screenshots captured from the installed applications in August 2026. Section 5 compares them, Section 6 states which features LoadWatch adopts, and Section 7 carries the findings forward.

2. Competitive Analysis: ESP "EskomSePush"

2.1 Overview

ESP has been on Google Play since February 2015 and reports over ten million installs at roughly 4.6 stars from more than 240 000 reviews [3], [6]. Its purpose is to tell a household whether supply is on and when that changes. The audience is the general public; the use case is planning the next few hours. Core functionality is saved areas, push alerts and a per-area community feed [4].

2.2 Strengths and Weaknesses

Strengths: Area granularity is decisive: a saved area resolves to a named suburb inside a named municipality, removing the largest source of user error. The area screen aggregates power, water and internet into one plain verdict, then offers two buttons, All good and Report issue. That binary confirmation is the innovative feature, because it turns a question that would need a sentence into one tap and yields data a backend can count.

Weaknesses: Scope has drifted into water, safety and Lost and Found, diluting the electricity purpose. Posts are free-form, so quality varies. No countdown is shown, only slot times; the warning lead is fixed rather than configurable; and nothing reports how accurate an area's schedule has been, so community signal is collected but never aggregated into visible trust.

2.3 Technical Implementation Theory

What follows is a hypothesis inferred from observed behaviour, store metadata and the vendor's published API, not a claim about the source code. ESP is very likely native Kotlin. The area screen's rounded cards, chips and pill buttons are consistent with Material 3 and therefore with Jetpack Compose, while older list screens may remain XML views, so a mixed codebase is the most plausible reading. State management points to MVVM: the verdict card, the confirmation buttons and the category feed all update together after a refresh, which suggests one observable state object exposed by a single ViewModel rather than several independent widgets. Content renders immediately on launch and then refreshes, the signature of a cached read followed by a network write, so a repository mediating Room over SQLite is the working assumption. Push delivery is almost certainly Firebase Cloud Messaging, since stage alerts arrive with the application closed. The backend is REST returning JSON: the vendor sells a documented API exposing area search, schedules and status [5].

2.4 Interface Architecture

The screenshots show a three-level hierarchy. Home lists saved areas under a national status banner; the area screen carries the verdict, the confirmation buttons and a categorised feed; notifications are per-area highlights naming the street. Adding an area is search-first.

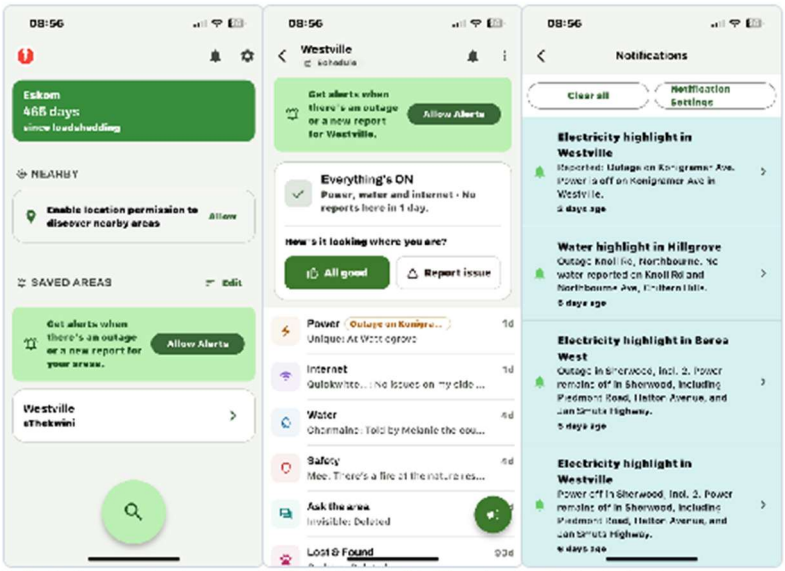


Fig 1a. Home: national banner, Nearby, Saved Areas

Fig 1b. Area: verdict, All good / Report issue, feed

Fig 1c. Notifications: per-area highlights

Figure 1. ESP "EskomSePush", captured 24 August 2026 on the Westville, eThekwini area.

3. Competitive Analysis: Load Shedding Notifier

3.1 Overview

Load Shedding Notifier, published by abisoft, covers Eskom and municipal areas nationwide. Its purpose is narrower than ESP's: publish national status and the schedule for the areas a user adds, then alert on change. The audience is anyone who wants the timetable rather than a community. Core functionality is a dashboard, per-area schedule tabs, a notices archive and an ad-supported free tier [7], [8].

3.2 Strengths and Weaknesses

Strengths: The dashboard states national status and the daily outlook in words. The area tab groups slots by day for several days ahead with a stage badge on each, so a user plans a week rather than an evening. The notices archive preserves each Eskom three-day outlook with a type filter. Quiet times, configurable alerts, low data usage and an explicit offline indicator suit expensive data and a device that loses power. Scope discipline is its real strength.

Weaknesses: Advertising occupies the bottom of every screen. The schedule is a dense date-stamped list with no countdown. Typography and colour are dated. Per-area tabs across the top do not scale past a few areas. Above all there is no path for a user to report that reality differs from the published schedule.

3.3 Technical Implementation Theory

The visual language suggests XML layouts rather than Compose: fixed-height rows, a tab strip and a classic action bar are cheap in the View system and unusual in a new Compose build. Tabs over a ViewPager2 with one Fragment per area is the most plausible structure, which also explains why the design degrades as areas are added. Architecture is probably MVVM or the older MVP, with a ViewModel per tab exposing schedule state, because collapsing a card preserves state across tab switches. Persistence is almost certainly SQLite since a month of schedule data is available and the stated low data usage implies the schedule is fetched once and read locally thereafter. Background refresh is likely WorkManager on a periodic request, with local notifications raised when a fetched status differs from the stored one; quiet times and lead settings fit SharedPreferences or DataStore. Networking is a thin Retrofit client returning small JSON payloads, and the free tier is monetised through an embedded AdMob banner view.

3.4 Interface Architecture

Navigation is two tabs, Dashboard and one per saved area, with the notices archive behind the bell. The screenshots show the national summary, a multi-day schedule with stage badges, and the archived three-day outlooks that give the app its historical value.

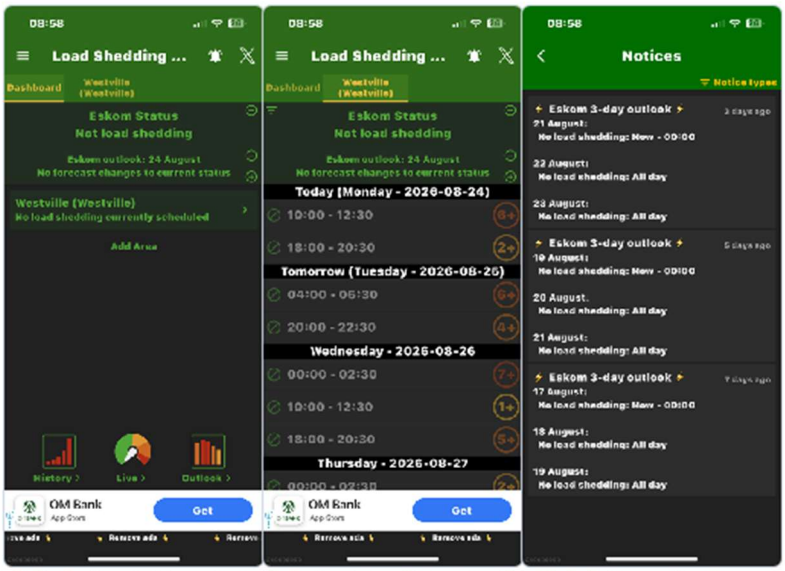


Fig 2a. Dashboard: status, outlook, Add Area

Fig 2b. Area tab: multi-day slots with stage badges

Fig 2c. Notices: archived Eskom three-day outlooks

Figure 2. Load Shedding Notifier, captured 24 August 2026 on the Westville area.

4. Competitive Analysis: My Smart City

4.1 Overview

My Smart City, by Acumen Software, is a citizen-to-municipality reporting platform rather than a load-shedding tool; the screenshots are its eThekwini instance. Its purpose is to let residents log municipal faults and follow them to resolution. The audience is ratepayers and the main use case is reporting a fault. Core functionality is a map, logged reports, a social feed and service bookings [9].

4.2 Strengths and Weaknesses

Strengths: Home carries an Eskom card with an explicit last-updated timestamp, a trust behaviour neither competitor matches. Log Reports places No Power, No Water and Pothole one tap from launch, and the map adds Quick Log and SOS. Local Insights charts in-progress against resolved reports, so a submission visibly enters a lifecycle. That accountability loop is the innovative feature. A categorised feed carries municipal announcements.

Weaknesses: Breadth dilutes the utility purpose: tutoring, cleaning and airtime sit beside outage information. Some content is badly stale, including an alert last updated 1 350 days ago. Layout defects are visible, with overlapping month labels on the insights chart and a truncated announcement label. The product is network dependent with no offline story, which is exactly wrong for a power outage.

4.3 Technical Implementation Theory

The same build ships on Android, iOS and Huawei AppGallery with identical typography, spacing and iconography, and the iOS package is large, so a cross-platform toolkit, Flutter or React Native, is a stronger hypothesis than two native codebases; a single Dart or JavaScript state container would then replace Android ViewModels. The map is a licensed SDK rather than a custom renderer, given clustered labelled points and a native locate control. The backend is REST over a workflow engine: a report has a status that advances, which implies server-owned state machines and a jobs table rather than simple CRUD. Photographs attached to reports imply object storage with the client uploading and persisting only a URL. An account is required, so requests carry a bearer token, and alerts arrive with the application closed, so push is registered per device and per municipality. The 1 350-day-old alert points to server content with no expiry rule, a data-governance defect rather than a client one.

4.4 Interface Architecture

Navigation is a five-item bottom bar: Home, Map, Create, Social Feed, Account, with a persistent Report Issues and Book Service sheet. The screenshots show the outage-relevant surfaces, the insights chart, the reporting map and the announcement feed.

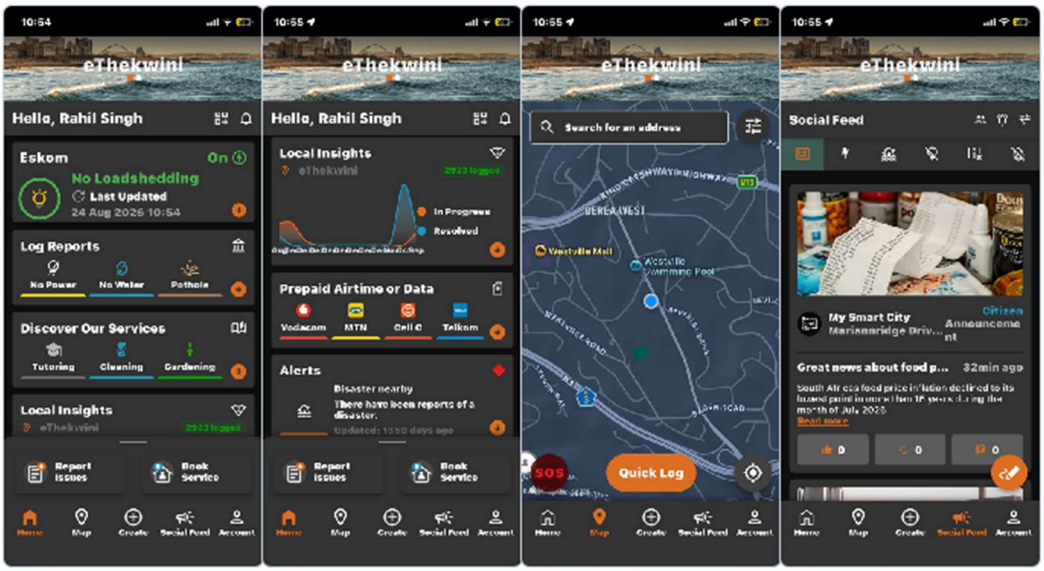


Fig 3a. Home: Eskom card, Log Reports | Fig 3b. Local Insights and stale alert | Fig 3c. Map: Quick Log and SOS | Fig 3d. Social Feed: announcements

Figure 3. My Smart City (eThekwini), captured 24 August 2026.

5. Comparison Matrix

The three converge on four points and diverge on three. **Similarities:** all are area-anchored rather than account-anchored, all treat push notification as the primary delivery mechanism, all let a user follow more than one location, and all place a status summary at the default destination. **Differences:** only ESP and My Smart City accept user input, and in incompatible ways, one a one-tap confirmation that is counted nowhere the user can see, the other a tracked ticket with a visible outcome; only Load Shedding Notifier treats offline operation and notification configuration as first-class; and none of the three converts a schedule into remaining time. LoadWatch occupies the space this reveals: *structured* confirmation with a *visible lifecycle* on an *offline-first* schedule.

Criterion	ESP "EskomSePush"	Load Shedding Notifier	My Smart City	LoadWatch decision
Primary purpose	Schedule plus community	Schedule plus alerting	Civic issue reporting	Outage truth for one area
Area granularity	Strong: real suburb names	Strong: nationwide municipal coverage	Medium: municipality scoped	Suburb level with supplier context
Default destination	Single-area dashboard	Multi-area list	Interactive map	Single-area dashboard with countdown
Live countdown	No: slot times only	No: outlook strip only	No	Yes: primary Part 2 feature
Notifications	Strong but fixed 55 min lead	Strong: configurable, quiet times	Alerts and emergency mode	FCM with user-set lead time
Multiple areas	Yes: home, work, family	Yes: configurable list	Multiple properties	Favourites plus side-by-side compare
User can contribute data	Yes: one-tap confirm plus chat	No path at all	Yes: formal tracked ticket	Structured two-state confirmation
Feedback on submission	None: confirmation has no visible outcome	Not applicable	Status timeline to resolution	Queued, accepted, reflected in score
Data trust signal	None: schedule stated as fact	None	Implicit via ticket status	Per-area accuracy score
Offline	Cached render, not	Explicit offline	Weak: network	Room-backed,

behaviour	surfaced	indicator, low data	dependent	offline state surfaced
Historical view	24h report graph	Month-ahead schedule	Own logged history	Past-month outage calendar
Localisation	English led	English led	English led	English, isiZulu, Afrikaans
Authentication	Account for chat	Largely anonymous	Account required	SSO plus biometric re-entry
Scope discipline	Weak: water, potholes, safety	Strong: single purpose	Weak: services, transport	Deliberately narrow
Hypothesised stack	Kotlin, mixed XML/Compose, MVVM, REST plus realtime	Kotlin, XML views, local SQLite, WorkManager	Cross-platform, map SDK, workflow backend, blob media	Kotlin, Compose, MVVM, Room, ASP.NET Core REST

Table 1. Comparison matrix. Green marks a demonstrated strength, amber a partial capability, red an absent one; the final column records the design decision each row produces. Rows 4, 7, 8, 9 and 10 are the differentiating gaps.

6. Feature Adoption Strategy

Features are prioritised by value created per unit of engineering effort. Priorities 1 to 4 are Part 2 scope; 5 to 8 are Final POE scope, which is why the design document isolates them behind interfaces.

1. **Area-anchored schedule dashboard (from ESP).** Resolving to a real suburb with visible supplier context removes the dominant onboarding failure [3], and forces Area to be a server-owned entity with a search endpoint.
2. **Live countdown (LoadWatch original; absent from all three).** No comparator converts a timestamp into remaining time [3], [7], [9]. Computed on-device from server-supplied ISO-8601 boundaries, it costs one field and no extra requests.
3. **Structured community confirmation (from ESP, given a lifecycle from My Smart City).** ESP proves demand for a one-tap signal; My Smart City proves a submission needs a visible outcome. LoadWatch sends a two-state event, POWER_OFF or POWER_BACK_ON, carrying area, user, timestamp and sync status, which the backend aggregates into a per-area accuracy score.
4. **Favourites and side-by-side comparison (from ESP and Load Shedding Notifier).** Both prove multi-location demand; comparing next-event times across saved areas on one screen is the increment neither provides, at the cost of one endpoint.
5. **Configurable notification lead time (from Load Shedding Notifier).** ESP's fixed warning is a documented complaint [3]. Storing lead time in UserSettings and applying it server-side makes the alert a contract rather than a broadcast.
6. **Offline-first schedule with synchronisation (from Load Shedding Notifier).** Its offline indicator and low-data posture suit expensive data and a device that loses power [7], [8]; Room is the recommended abstraction over SQLite [12], and offline sync is a Final POE requirement [1].
7. **Stage-change alert with recalculation (from ESP).** LoadWatch makes this a backend event that recalculates affected schedules and publishes through FCM [14], keeping recalculation authoritative and the client thin.
8. **Historical outage calendar (from Load Shedding Notifier and My Smart City).** Combining past events with past confirmations produces what neither has: a visible record of how often the published schedule matched reality.

Two features are deliberately **rejected**: free-form chat, which yields unstructured data and a moderation obligation the team cannot meet; and broad civic service listings, since breadth is the shared weakness of the largest comparators.

7. Conclusion

The research establishes that LoadWatch should not compete by publishing another timetable. ESP shows that area accuracy, proactive alerting and community signal create the category's value and that breadth erodes it.

Load Shedding Notifier shows that offline resilience and user-owned notification configuration are product features, and that a schedule with no correction path cannot defend its accuracy. My Smart City shows that a submission must have a visible lifecycle to be worth making. Together they identify one unmet need: a narrow, offline-capable application in which the published schedule and observed reality are reconciled in public and the difference between them is measured.

This directly establishes the Planning and Design document, which adopts this scope, turns the eight adopted features into numbered requirements with Part 2 and Final POE scope separated, applies the interface lessons, specifies the REST API implied by Priorities 1 to 4, fixes the modular architecture that makes Final POE features additive, defines the data models, and schedules the engineering work.

References

[1] Android Developers, "Build an offline-first app." [Online]. Available: https://developer.android.com/topic/architecture/data-layer/offline-first. Accessed: Aug. 24, 2026.

[2] Android Developers, "Jetpack Compose." [Online]. Available: https://developer.android.com/develop/ui/compose. Accessed: Aug. 24, 2026.

[3] Google Play, "ESP 'EskomSePush'," wellwellwell. [Online]. Available: https://play.google.com/store/apps/details?id=com.ashwhale.sepush.eskom. Accessed: Aug. 23, 2026.

[4] EskomSePush, "ESP: South Africa's #1 Loadshedding and Community App." [Online]. Available: https://esp.info/. Accessed: Aug. 23, 2026.

[5] EskomSePush, "EskomSePush API Subscription: Documentation v3.0 and v3.1." [Online]. Available: https://eskomsepush.gumroad.com/l/api. Accessed: Aug. 23, 2026.

[6] AppBrain, "ESP 'EskomSePush': Android app statistics, version history and ratings." [Online]. Available: https://www.appbrain.com/app/esp-eskomsepush/com.ashwhale.sepush.eskom. Accessed: Aug. 23, 2026.

[7] Google Play, "Load Shedding Notifier," abisoft. [Online]. Available: https://play.google.com/store/apps/details?id=com.abisoft.loadsheddingnotifier. Accessed: Aug. 23, 2026.

[8] Apple App Store, "Load Shedding Notifier," abisoft: feature list and version release notes. [Online]. Available: https://apps.apple.com/za/app/load-shedding-notifier/id1560357687. Accessed: Aug. 23, 2026.

[9] Acumen Software, "My Smart City: Mobile Smart City Solution," product site and feature pages. [Online]. Available: https://www.mysmart.city/. Accessed: Aug. 23, 2026.

[10] Apple App Store, "MySmart.City," Acumen Software (Pty) Ltd. [Online]. Available: https://apps.apple.com/za/app/mysmart-city/id1555236570. Accessed: Aug. 23, 2026.

[11] Android Developers, "Guide to app architecture." [Online]. Available: https://developer.android.com/topic/architecture. Accessed: Aug. 23, 2026.

[12] Android Developers, "Save data in a local database using Room." [Online]. Available: https://developer.android.com/training/data-storage/room. Accessed: Aug. 23, 2026.

[13] Firebase, "Authenticate with Google on Android." [Online]. Available: https://firebase.google.com/docs/auth/android/google-signin. Accessed: Aug. 23, 2026.

[14] Firebase, "Set up a Firebase Cloud Messaging client app on Android." [Online]. Available: https://firebase.google.com/docs/cloud-messaging/android/client. Accessed: Aug. 23, 2026.

[15] Android Developers, "Show a biometric authentication dialog." [Online]. Available: https://developer.android.com/identity/sign-in/biometric-auth. Accessed: Aug. 23, 2026.

[16] A. Walker, "3 Android load shedding apps for South Africa," Memeburn, Nov. 2018. [Online]. Available: https://memeburn.com/2018/11/best-load-shedding-apps-south-africa/. Accessed: Aug. 23, 2026.

[17] Microsoft Learn, "Tutorial: Create a controller-based web API with ASP.NET Core." [Online]. Available: https://learn.microsoft.com/en-us/aspnet/core/tutorials/first-web-api. Accessed: Aug. 23, 2026.

[18] Joburg ETC, "City Power warns of planned outages across Johannesburg next week," Aug. 2026. [Online]. Available: https://www.joburgetc.com/news/city-power-planned-outages-joburg-next-week/. Accessed: Aug. 23, 2026.